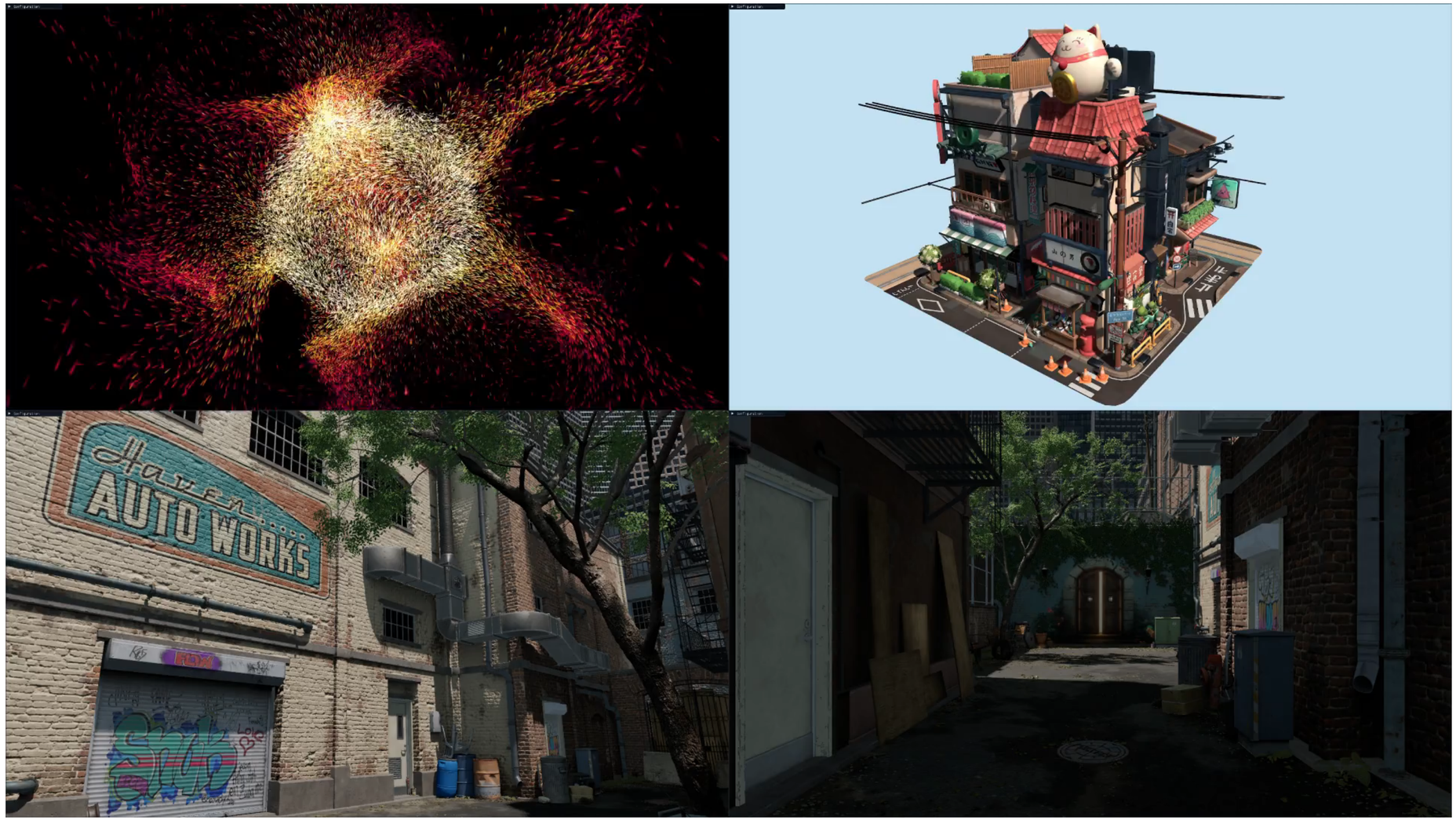
Crat3r
#00 - Organisation

Erik Eismann, Marc Stamminger

October 13, 2026

What is CRAT3R?



What is CRAT3R?

- 10 ECTS Masters Project
- 4 Vulkan programming assignments (50% of final grade)
- Small final project + report & presentation (50% of final grade)
- Broad topics:
  - Vulkan GPU programming and common patterns
  - Deep dive into select computer graphics concepts

Goals of CRAT3R

At the end of this course you should

- be able to use the Vulkan API for graphics applications.
- have a better understanding of how GPUs function.
- have deepened your knowledge of modern computer graphics.

Things to look out for

- Crat3r is not an introduction to computer graphics. Prior knowledge of computer graphics, e.g. the render pipeline, shader programming and lighting calculations is expected.

- Crat3r is not the easiest way to get your ECTS (similar to GraPra). Expect to spend lots of hours every week on the programming assignments and factor in the final project and report.

Lecture

- Tuesday 14:15 - 15:45, Room 01.151-128

Date	Topic
Oct. 13	Introduction to Vulkan and GPU-Debugging
Oct. 20	Buffers and Memory, GPU Compute and Descriptors
Oct. 27	Images, Samplers and the Swapchain
Nov. 03	Renderpasses and Rasterization
Nov. 10	Subpasses and Attachments
Nov. 17	Instance, Devices and Queues
Nov. 24	Modern Vulkan Features
Dec. 01	Vulkan in Game Engines
Dec. 08	Hardware Accelerated Ray Tracing
Dec. 15	Practial Anti-Aliasing, Frame-gen and Upscaling

Programming Assignments

- Complex programming assignments over multiple weeks
- Students work in groups of two (form groups yourself, individual submissions not accepted)
- Submission of changed source files via StudOn. Teams handle version control themselves
- Short presentations (only your group, no specific format) of results with follow-up questions
- In CIP-Pool (00.153-113) on Thursdays 9:00 - 12:00. Q&A on weeks without submissions
- Free-form final project with (public) presentation and written report

Start	Submission	Presentation	Topics
Oct. 13	Nov. 02	Nov. 05	Buffers, Images, Descriptors and Compute Shaders
Nov. 03	Nov. 23	Nov. 26	Rasterization, Renderpasses and Attachments
Nov. 24	Dec. 14	Dec. 17	Lighting, Shadows and Anti-Aliasing
Dec. 15	Jan. 11	Jan. 14	Hardware Accelerated Ray Tracing
Jan. 12	TBA	TBA	Final Project

Programming Assignments

LGDV
COMPUTER GRAPHICS • ERLANGEN
Department of Computer Science

Erik Eismann, Marc Stamminger *Erlangen, xx.xx.2026*

Compute Rasterization And Tracing For 3D Rendering (exercise sheet 1)

Welcome to the first exercise of CRAT3R - Compute Rasterization And Tracing For 3D Rendering. In this exercise, you will learn how to implement a simple GPU particle system using compute shaders. To achieve this, you will discover how to create buffers and images in Vulkan, bind them to shaders using descriptors and create and dispatch a compute pipeline.

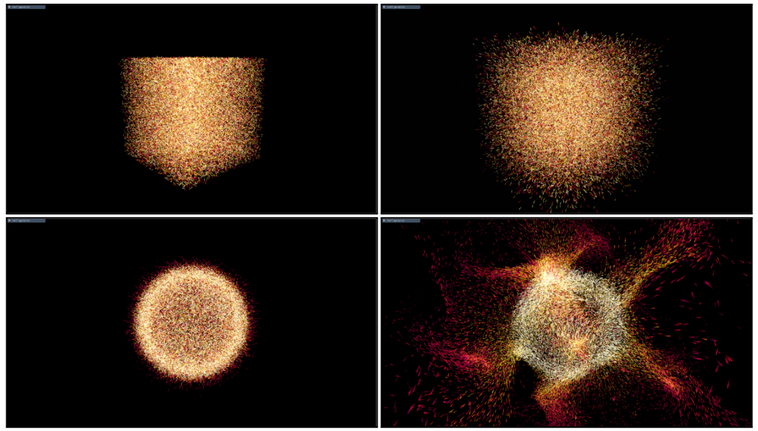


Figure 1: Rendered results of this exercise as you progress through the assignments.

Some important tips for the assignments:

- **Plan a significant amount of time for the assignments**. Start early, plan ahead and decide what to do and when. For the first exercise, aim to complete at least three sub-assignments per week.
- **Beware the validation layers!** They are your friend, telling you when your implementation doesn't make sense. Always fix all warnings from the validation layers before moving on to the next task. Note: they are only enabled by default when compiling in Debug mode.
- **Prepare well for your submission.** You are expected to present your results and highlight what you have learned. If you can explain where things went wrong and how you resolved the issues, you are on the right track. It is not enough to simply complete the tasks; you should also be able to explain the reasons behind the code you have written.

LGDV
COMPUTER GRAPHICS • ERLANGEN

Assignment 1.0 Initial Setup

First, download and extract the code from StudOn. To build the project, you will need a compiler with C++20 support, CMake, as well as the Vulkan SDK installed on your system. For Windows download the SDK from `https://vulkan.lunarg.com/`. For Linux install the relevant packages (e.g. on Ubuntu: `libvulkan-dev`, `vulkan-utility-libraries-dev`, `vulkan-validationlayers`, and optionally `vulkan-tools`). The `glslc` shader compiler is also required. If you download the full SDK from the website, it should come packaged with it. Otherwise, you will need to install it separately. Then, ensure that it is on your shell's search path.

To build the project, first run `cmake -S . -B build -G Ninja` on Linux, or on Windows configure the project using the CMake GUI. Then, either run `cmake --build build` or use Visual Studio to compile. If there are no errors, you should then find the `a01` executable in the `build` directory. Running this should open a window with a green background and a small UI on top.

Next, take a look at the project layout. All your implementation will be done in the files `main.cpp` and `particles.comp`. Much of the initialization work for Vulkan has already been done for you. The crat3r core library initiates Vulkan, selects and creates a device, and creates a window, surface and swapchain. A particle renderer is also provided for the first assignment. You don't need to understand what it does for now; it's just there to help you see something on your screen right after you add the necessary storage buffer.

Assignment 1.1 Storage Buffer Creation

First, we need a storage buffer to hold the particles on the GPU. Refer to `particle.h` to see what data is required for each particle. Then, start your implementation of `main.cpp` by allocating a storage buffer and filling it with some initial data. As we will be creating multiple buffers throughout this assignment, the first step is to implement the `createBuffer` function, which creates a buffer and allocates memory for it. Next, use this function in `createStorageBuffer` to create the storage buffer, and then map the memory to upload data to it. If you do this correctly, you should see a cube of particles similar to the top-left image of Figure 1.

A note on performance: Allocating a new block of memory for each buffer is very inefficient. Additionally, the minimum size and maximum number of allocations are typically restricted. However, as we will only be allocating a few buffers in this assignment, we will stay well below these limits. Therefore, we will not yet use a dedicated memory allocator in this assignment, but stick with `vkAllocateMemory`.

Assignment 1.2 Staging Buffer

While the first approach of creating a host-visible storage buffer is practical, it is rather inefficient. Since we will never access the buffer from the CPU again after the initial upload, it would be better to

Programming Assignments

Exercise Presentation

- Short presentation of your findings (10-15 min)
- Any format you find appealing (code review, screenshots, slides, …)
- Explain what had to be done in the assignment and focus on key takeaways and hurdles (e.g. ‘if you set the wrong config option here, this weird things happens…’)
- Assume level of knowledge right before completing the assignment
- Small number of follow-up questions after the presentation
- Both team members must have equal share of conversation
- No public audience, only your team is present
- Link for registration for time slots (not yet) on StudOn
- Absence requires doctors note

A Note on AI

- We can not stop you from using AI for the assignments.
- From experience: strong use will stop you from learning anything.
- You have to explain the key takeaways and difficulties during your presentation and be able to answer questions about any single line of code you (or an LLM) wrote.
- Only your presentation and answers are graded, not your code.

Registration and Groups

- At the end of this lecture, come forward to lock in your spot.
- Also name your group partner for the first assignment.
  (You can change partner for later assignments if you want.)

Feedback and Questions

- You are my beta testers. :)
- Please give feedback at the end, but also during the course.
- Questions now?